

BITÁCORA DE PROYECTO

Flight Price Alert Web

"Somos Rata" — Cómo construí un MVP de alertas de vuelos con IA, desde cero hasta producción

Documento de aprendizaje paso a paso: la historia completa, las decisiones, los errores y cómo se resolvieron. Pensado para replicar el proyecto y para construir nuevas aplicaciones sobre la misma base.

Autor del proyecto: Daniel

Herramientas: Claude (chat) + Claude Code + Python + Streamlit

Costo de las APIs usadas: gratuito (tiers free / test)

Fecha del documento: julio 2026

Contenido

1. Qué es este documento y cómo usarlo

2. Visión general del proyecto: qué construimos y por qué

El problema, la solución, la arquitectura en 3 planos

3. Fase 1 — Diseño y generación del MVP (en el chat de Claude)

Arquitectura, modelo de datos, generación del código, prueba local

4. Fase 2 — Correr el proyecto en local (VS Code / Windows)

Errores reales y cómo se resolvieron

5. Fase 3 — El salto a Claude Code

6. Fase 4 — Base de datos en la nube (Supabase)

7. Fase 5 — Subir a GitHub y desplegar en Streamlit Cloud (el link público)

8. Fase 6 — Conectar las APIs reales (Duffel, SendGrid, Twilio)

9. Fase 7 — Marca propia: logo, nombre "Somos Rata"

10. Fase 8 — Mejoras por feedback de usuarios (buscador por ciudad)

11. Checklist de seguridad (lo que aprendí a las malas)

12. Cómo replicar este proyecto desde cero (receta condensada)

13. Cómo crear una app NUEVA con la misma base (plantilla reutilizable)

14. Cómo escalar: de MVP a aplicación real (app store, WhatsApp propio, correo propio)

15. Cómo monetizar cuando el MVP ya tiene usuarios

16. Glosario rápido

1. Qué es este documento y cómo usarlo

Este documento es la **bitácora completa** de cómo construí "Flight Price Alert Web" (marca "Somos Rata"), una aplicación web donde cualquier persona deja una alerta de vuelo y recibe aviso por email o WhatsApp cuando el precio baja de su objetivo.

Tiene dos propósitos:

- **Aprender y replicar.** Está escrito paso a paso, como para un estudiante que quiere rehacerlo desde cero, con los enlaces de cada herramienta y las decisiones explicadas.
- **Construir hacia adelante.** Las secciones finales muestran cómo usar esta misma base para crear apps nuevas (alertas de tasas hipotecarias, precios de viviendas, etc.), cómo escalar a una app "de verdad" y cómo monetizarla.

Cómo leer las fases. El proyecto se hizo en dos herramientas: primero conversando con **Claude en el chat** (diseño y generación del código), y luego con **Claude Code** (conexiones reales, despliegue e iteración). Cada fase indica en qué herramienta ocurrió.

2. Visión general: qué construimos y por qué

El problema

Comparar precios de vuelos a mano es tedioso: hay que revisar muchas fechas y rutas una y otra vez. La idea fue automatizar eso: el usuario dice "quiero ir de Santiago a Madrid, máximo 500 USD" y el sistema vigila el precio por él, avisándole cuando aparezca una oportunidad.

La arquitectura en tres planos (la decisión más importante)

El sistema se dividió en **tres partes independientes**, unidas solo por la base de datos. Entender esto es la clave de todo el proyecto:

Plano	Qué hace	Dónde vive
1. Web pública	El formulario donde la gente crea alertas y los dashboards	Streamlit Cloud
2. Base de datos	Guarda usuarios, alertas, historial de precios y notificaciones (la "fuente de verdad")	Supabase (PostgreSQL)
3. Worker automático	Cada 6 horas revisa precios, compara con el objetivo y notifica	GitHub Actions (cron)

¿Por qué separar el worker de la web? Porque Streamlit Cloud "duerme" la app tras la inactividad. Si el chequeo de precios dependiera de que la web esté despierta, fallaría. Por eso el worker corre solo, en otro servidor (GitHub Actions), y ambos se comunican **solo** a través de la base de datos. Esta decisión es la que hace que el sistema funcione 24/7 sin que tu PC esté encendido.

Principio de diseño clave: "degradación elegante"

El proyecto se diseñó para funcionar **sin ninguna credencial** al principio (modo demo):

- Sin base de datos en la nube → guarda en archivos CSV locales.
- Sin API de vuelos → usa un generador de precios simulado ("mock").
- Sin SendGrid/Twilio → escribe en consola lo que "habría enviado" (modo dry-run).

Esto permitió mostrar el proyecto completo funcionando desde el minuto cero, e ir conectando los servicios reales uno por uno, sin frustración.

3. Fase 1 — Diseño y generación del MVP

Herramienta: chat de Claude

Antes de escribir una sola línea de código, se definió la arquitectura y se pidió aprobación. Este orden (diseñar → aprobar → generar) evita rehacer trabajo.

Los pasos de esta fase

1. **Definir arquitectura:** los tres planos, decisiones técnicas justificadas, diagrama de flujo.
2. **Definir el modelo de datos:** cuatro tablas — `users`, `alerts`, `price_history`, `notifications`.
3. **Generar el código completo,** archivo por archivo, comentado y probado.
4. **Probar en el entorno de Claude** el modo mock + CSV, para entregar un ZIP verificado.

Estructura del proyecto generado

```
flight-price-alert-web/
├── app_streamlit.py          # Web pública (formulario)
├── check_prices.py          # Worker: revisa precios y notifica
├── ejecutar_todo.py        # Script demo: siembra datos de ejemplo
├── src/
│   ├── flight_api.py       # Selector de proveedor (mock/duffel)
│   ├── mock_flight_api.py  # Proveedor simulado (default)
│   ├── database.py         # Supabase + fallback CSV
│   ├── alert_rules.py      # Umbral de precio + anti-spam
│   ├── notifier_email.py   # SendGrid
│   ├── notifier_whatsapp.py # Twilio
│   ├── city_resolver.py    # (agregado después) ciudad -> código IATA
│   └── utils.py            # get_secret dual, validaciones
├── pages/
│   ├── 01_Admin_Dashboard.py # KPIs y tablas
│   └── 02_Price_History.py   # Series de tiempo de precios
├── .streamlit/config.toml   # Identidad visual
├── .github/workflows/check_prices.yml # Cron cada 6 h
└── docs/                   # architecture, data_model, deployment_guide
```

Concepto que aprendí: el "patrón Strategy". En `flight_api.py` hay un único punto de entrada `search_flights()`, y una variable `FLIGHT_PROVIDER` decide si usa el mock o Duffel. Cambiar de simulado a real es cambiar una variable, no reescribir el código.

4. Fase 2 — Correr el proyecto en local (VS Code, Windows)

Herramienta: VS Code + terminal

Aquí aparecieron los primeros errores reales. Documento cada uno porque volverán a pasar en cualquier proyecto Python en Windows.

Los comandos correctos (mi flujo en Windows)

```
python -m pip install -r requirements.txt
python ejecutar_todo.py          # genera datos de demo
python -m streamlit run app_streamlit.py # abre la web en el navegador
```

La web se abre en `http://localhost:8501`. El dashboard NO está en VS Code: está en el **navegador**, en el menú lateral izquierdo (páginas "Admin Dashboard" y "Price History").

Error 1: "source no se reconoce"

Causa: `source .venv/bin/activate` es de Mac/Linux. En Windows no existe.

Solución: en Windows la activación es `.venv\Scripts\activate` (con backslash y "Scripts", no "bin"). O, más simple para empezar, no usar entorno virtual y correr con `python -m pip` y `python -m streamlit` directamente.

Error 2: "Could not install ... Long Path" (al instalar twilio)

Causa: Windows limita las rutas a 260 caracteres. El proyecto estaba en una carpeta muy profunda dentro de OneDrive, y las carpetas internas de `twilio` se pasaban del límite.

Solución (la que recomiendo): mover el proyecto a una ruta corta, tipo `C:\dev\flight-price-alert-web`, y fuera de OneDrive (OneDrive da problemas con Python y Git). Alternativas: activar "Long Paths" en Windows con permisos de administrador, o saltarse temporalmente la librería que falla.

Lección general: guarda tus proyectos de código en una ruta corta y fuera de OneDrive. Ejemplo bueno: `C:\dev\mi-proyecto`. Te ahorra estos dos errores de una.

Detalle útil: autoguardado en VS Code

Menú **File** → **Auto Save**. Con esto, VS Code guarda solo. Recordar: guardar no re-ejecuta. Streamlit sí ofrece un botón "Rerun" (o "Always rerun") cuando detecta cambios; los scripts normales hay que volver a lanzarlos a mano.

5. Fase 3 — El salto a Claude Code

Herramienta: Claude Code (dentro de VS Code)

El chat fue ideal para diseñar y generar. Pero conectar credenciales reales, ejecutar, ver el error y ajustar —el ciclo "corre, mira, arregla"— se hace mucho mejor en **Claude Code**, que ve tus archivos, ejecuta comandos en tu terminal y lee los errores directamente.

Cómo se instala

- **Opción A (más fácil):** la app de escritorio de Claude, que trae una pestaña "Code".
- **Opción B:** la extensión oficial "Claude Code" en VS Code (icono de Extensiones → buscar "Claude Code" → instalar → iniciar sesión).

Confusión que tuve: el panel de chat genérico de VS Code (que decía "Language model unavailable") NO es Claude Code. Claude Code es un producto aparte que hay que instalar e iniciar sesión. Señal de que estás en el correcto: dice "Claude Code" y responde.

Cómo funciona el permiso por comando

Claude Code nunca ejecuta nada sin tu aprobación. Ante cada comando muestra "Allow this command?" con opciones Yes / No. Aprendí a leer los **diffs** antes de aprobar:

- **Verde con +** = línea que se agrega.
- **Rojo con -** = línea que se quita.
- Sin color = contexto que no cambia.

Regla de seguridad que adopté: para leer el archivo `.env` o para editar `app_streamlit.py` (mi web pública), elijo siempre "Yes" (solo esta vez), no "allow all edits this session". Un poco más de fricción a cambio de ver cada cambio en lo que ya está en producción.

6. Fase 4 — Base de datos en la nube (Supabase)

Herramienta: Supabase (navegador) + Claude Code

El modo CSV local sirve para demo, pero no para producción: en Streamlit Cloud el CSV se borra y el worker (en otro servidor) no lo ve. Por eso Supabase es **obligatoria** para el link público.

Enlace: <https://supabase.com> (tier gratuito).

Pasos

1. **Crear proyecto** en Supabase (se entra con GitHub).
2. **Obtener credenciales:** en **Settings** → **API**. El **Project URL** es tu `SUPABASE_URL`. En **API Keys**, copiar la **Publishable key** (equivalente a la antigua `anon public`) como `SUPABASE_KEY`. **Nunca** usar la `secret` / `service_role` en un proyecto compartible.
3. **Crear el archivo** `.env` (copia de `.env.example`) y pegar ambos valores. Cuidar el nombre exacto: `SUPABASE_URL` y `SUPABASE_KEY`, sin espacios.
4. **Crear las tablas:** en el **SQL Editor** de Supabase, pegar y ejecutar (Run) todo el bloque SQL que está en `docs/data_model.md`.
5. **Desactivar RLS** (Row Level Security) en las 4 tablas, para que la app pueda leer/escribir con la publishable key. Para un MVP de portafolio es aceptable:

```
alter table users disable row level security;
alter table alerts disable row level security;
alter table price_history disable row level security;
alter table notifications disable row level security;
```

Cómo verificar la conexión

```
python -c "from src.database import Database; db = Database(); print('Backend:', db.backend)"
```

Si imprime `Backend: supabase`, quedó conectada. Si dice `csv`, no está leyendo el `.env` (revisar nombre del archivo y de las variables).

Error que tuve: Claude Code decía que el `.env` seguía vacío aunque yo veía los valores en pantalla. **Causa:** no había guardado el archivo (la pestaña mostraba un punto = cambios sin guardar). Claude Code lee el disco, no lo que se ve en pantalla. **Solución:** `Ctrl + S`.

7. Fase 5 — Subir a GitHub y desplegar en Streamlit Cloud

Herramienta: Claude Code + GitHub + Streamlit Cloud

Paso previo que casi se me olvida: subir a GitHub

"Conectar la cuenta de GitHub" (dar permiso) NO es lo mismo que "subir el proyecto". Streamlit solo despliega código que ya está en GitHub. Le pedí a Claude Code que creara el repositorio y subiera todo, verificando antes que el `.env` estuviera en `.gitignore` (para no filtrar claves).

Desplegar en Streamlit Community Cloud

Enlace: <https://share.streamlit.io> (entrar con GitHub).

1. **New app** → usar el "interactive picker" (más fácil que pegar URL).
2. Configurar exactamente:
 - **Repository:** `tu-usuario/flight-price-alert-web`
 - **Branch:** `main` (no `master`)
 - **Main file path:** `app_streamlit.py` (con guion bajo)
3. **Advanced settings** → **Secrets:** pegar en formato TOML, ANTES de Deploy:

```
FLIGHT_PROVIDER = "mock"
SUPABASE_URL = "https://xxxx.supabase.co"
SUPABASE_KEY = "sb_publishable_xxxx"
NOTIFY_COOLDOWN_HOURS = "24"
```

1. **Deploy** → en 1-3 minutos entrega la URL pública (ej. `somosrata.streamlit.app`).

Errores que tuve en el formulario de Streamlit:

- "This repository does not exist": el repo era **privado** y Streamlit no tenía permiso. Solución: dar acceso a la GitHub App de Streamlit, o hacer el repo público (para portafolio conviene público, y como el `.env` no se sube, no expone secretos).
- "This branch does not exist": estaba escrito `master`; mi rama era `main`.
- "This file does not exist": estaba el ejemplo `streamlit_app.py`; el mío es `app_streamlit.py`.

Concepto clave del cron: el archivo `.github/workflows/check_prices.yml` hace que el worker corra cada 6 horas, pero **solo cuando el proyecto está en GitHub** (corre en los servidores de GitHub, no en tu PC). En local, el monitoreo es manual: corres `python check_prices.py` cuando quieras. Automático 24/7 = GitHub + Supabase.

8. Fase 6 — Conectar las APIs reales (todas gratis)

Herramienta: Claude Code + paneles de cada servicio

Todas las APIs se usaron en su nivel gratuito (free / test). Orden recomendado:

Proveedor de vuelos: Duffel (test)

Enlace: <https://duffel.com>. Se obtiene un `DUFFEL_ACCESS_TOKEN` (empieza con `duffel_test_`) y se pone `FLIGHT_PROVIDER=duffel`.

Nota importante de mi proyecto: originalmente el plan contemplaba Amadeus, pero su portal self-service se decomisiona en julio 2026 y dejó de aceptar cuentas nuevas, así que **migré a Duffel**. Cuando Duffel no expone un checkout público por oferta, uso un **enlace de referencia a Google Flights** como respaldo para que el usuario complete la compra.

Email: SendGrid (free)

Enlace: <https://sendgrid.com>. Se crea una API Key con permiso "Mail Send" (`SENDGRID_API_KEY`) y se verifica un remitente (Single Sender) que va en `SENDGRID_FROM_EMAIL`. Sin la key, el email corre en modo dry-run (no envía, solo registra).

WhatsApp: Twilio Sandbox (free)

Enlace: <https://twilio.com>. Da `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN` y `TWILIO_WHATSAPP_FROM`.

Limitación del Sandbox (no es un bug del código): cada destinatario debe primero enviar un código `join <palabra>` al número del sandbox de Twilio antes de poder recibir mensajes. Por eso el email es el canal confiable en la demo, y WhatsApp funciona para quien se une al sandbox. Escapar de esta limitación (WhatsApp con tu propio número y logo) se cubre en la sección de escalado.

9. Fase 7 — Marca propia: logo y nombre "Somos Rata"

Herramienta: Claude Code

Se agregó una carpeta `assets/` con el logo (`somos_rata_logo.png`) y una imagen para WhatsApp, y se integró la identidad visual (colores, nombre) en la web. Esto convierte un MVP genérico en un producto con cara propia, algo importante tanto para usuarios como para portafolio.

10. Fase 8 — Mejoras por feedback de usuarios

Herramienta: Claude Code

Los primeros usuarios reportaron que no sabían los códigos IATA de 3 letras. Solución: se creó `src/city_resolver.py`, un diccionario que traduce el **nombre de la ciudad** al código (ej. "Madrid" → MAD, "Paris" → PAR).

Concepto aprendido — códigos de ciudad: existen códigos IATA "metropolitanos" que agrupan todos los aeropuertos de una ciudad (PAR = CDG+ORY+BVA, LON = LHR+LGW+STN, NYC = JFK+LGA+EWR). Al pasar el código de ciudad a Duffel, busca en todos esos aeropuertos, y como el sistema ordena por precio, elige el más barato automáticamente. No hubo que programar esa lógica: la API lo hace.

Ampliación posterior: se agregaron todas las ciudades de Chile (Arica ARI, Iquique IQQ, Antofagasta ANF, Concepción CCP, etc.) y capitales de Latinoamérica. Para cobertura mundial futura, existe la API de "Places" de Duffel (autocompletado real), que se deja como mejora posterior.

11. Checklist de seguridad (lo que aprendí a las malas)

Regla	Por qué
El <code>.env</code> nunca va a GitHub	Contiene tus claves. Ya está en <code>.gitignore</code> .
No mostrar el <code>.env</code> en capturas de pantalla	Compartir una captura con claves visibles las expone; hay que rotarlas si pasa.
Usar la Publishable key, no la Secret	La secret tiene permisos totales sobre la base.
Credenciales en "Secrets" de Streamlit, no en el código	El repo público nunca debe contener claves.
Aprobar acceso al <code>.env</code> caso por caso en Claude Code	Control sobre quién toca tus secretos.
Consentimiento del usuario antes de crear alerta	Recibir avisos requiere permiso explícito.

Si alguna clave quedó expuesta (por ejemplo en una captura), lo correcto es **rotarla** (regenerarla) en el panel del servicio: Supabase, Duffel, SendGrid, Twilio, y cambiar el `ADMIN_PASSWORD`. Una clave expuesta se considera comprometida.

12. Cómo replicar este proyecto desde cero (receta condensada)

Si tuvieras que rehacerlo, este es el orden mínimo:

- Diseño** (chat de Claude): pedir arquitectura + modelo de datos, aprobar, generar el código.
- Local** (VS Code): carpeta en ruta corta fuera de OneDrive. `python -m pip install -r requirements.txt` → `python ejecutar_todo.py` → `python -m streamlit run app_streamlit.py`.
- Claude Code**: instalar, iniciar sesión, abrir la carpeta.
- Supabase**: crear proyecto, copiar URL + Publishable key al `.env`, ejecutar el SQL de las tablas, desactivar RLS.
- GitHub**: pedir a Claude Code que cree el repo y suba (verificando `.gitignore`).
- Streamlit Cloud**: New app → repo / main / `app_streamlit.py` → Secrets (TOML) → Deploy.
- APIs reales**: Duffel (vuelos), SendGrid (email), Twilio (WhatsApp). Cambiar `FLIGHT_PROVIDER` a `duffel`.
- Marca + mejoras**: logo, nombre, y ajustes según feedback.

13. Cómo crear una app NUEVA con la misma base

La gran ventaja: **la arquitectura es reutilizable**. La estructura de "usuarios dejan una alerta sobre un valor → un worker revisa periódicamente una fuente de datos → notifica cuando se cumple una condición" sirve para muchísimas cosas. Solo cambian dos piezas: **la fuente de datos** (qué API consultas) y **la condición** (qué comparas).

Ejemplos que mencionaste

App nueva	Qué cambia respecto a vuelos	Fuente de datos posible
Alertas de tasas hipotecarias	El "precio" es la tasa de interés; la condición es "tasa ≤ objetivo"	APIs/publicaciones de bancos, datos de bancos centrales, sitios de comparación
Alertas de precio de viviendas	El "precio" es el valor de un depto/casa en una zona; condición "valor ≤ objetivo"	Portales inmobiliarios con API, o datos públicos de tasaciones
Alertas de dólar / cripto	El "precio" es el tipo de cambio; condición por umbral	APIs financieras gratuitas

El prompt reutilizable para Claude Code

Para arrancar una app nueva, este prompt adapta el proyecto existente:

Tengo un proyecto base (Flight Price Alert Web) con esta arquitectura: web pública en Streamlit + base de datos Supabase + worker que corre cada 6 horas en GitHub Actions y notifica por email/WhatsApp cuando se cumple una condición de precio. Quiero crear una app NUEVA reutilizando la misma arquitectura, pero para [ALERTAS DE TASAS HIPOTECARIAS].

Mantén los tres planos y la degradación elegante (modo mock + fallback CSV). Cambia solo: (1) la fuente de datos: en vez de Duffel, consultar [FUENTE], (2) la condición de alerta: en vez de precio de vuelo, comparar [LA TASA] contra el objetivo del usuario, (3) los campos del formulario según el nuevo dominio. Primero muéstrame el plan y la nueva arquitectura antes de tocar código. Deja un proveedor mock por defecto para poder demostrarlo sin API real.

Por qué esto funciona: reutilizas todo lo difícil (base de datos, worker, cron, notificaciones, anti-spam, dashboards, despliegue) y solo reescribes la parte específica del dominio. Es la diferencia entre construir cada casa desde cero y tener los cimientos ya hechos.

14. Cómo escalar: de MVP a aplicación real

El MVP actual es una web (Streamlit) con servicios en modo free/test. Escalar significa quitar esas limitaciones. Aquí las rutas, en orden de dificultad.

14.1 WhatsApp propio, con tu logo, sin la limitación del "join"

El sandbox de Twilio exige el código "join" y usa un número compartido. Para tener WhatsApp profesional (tu número, tu nombre/logo, sin que el usuario tenga que unirse):

- Necesitas la **WhatsApp Business API** (a través de Twilio u otro proveedor, o directo con Meta).
- Requiere **verificar tu empresa** con Meta (Meta Business Manager) y registrar un número dedicado.
- Las **plantillas de mensaje** (templates) deben ser aprobadas por Meta para mensajes iniciados por ti.
- Tiene **costo por conversación** (ya no es free), pero es lo que permite escalar a muchos usuarios.

Enlaces: Twilio WhatsApp Business ([twilio.com/whatsapp](https://www.twilio.com/whatsapp)), Meta Business (business.facebook.com).

14.2 Correo con dominio propio (no @gmail)

Para enviar desde `alertas@somosrata.com` en vez de un Gmail:

- **Comprar un dominio** (ej. `somosrata.com`) en un registrador (Namecheap, GoDaddy, Cloudflare).
- En SendGrid, hacer **Domain Authentication** (agregar unos registros DNS que te da SendGrid a tu dominio). Esto mejora la entregabilidad y permite remitentes profesionales.
- Opcional: un buzón real (Google Workspace o Zoho Mail) si además quieres recibir correos en esa dirección.

14.3 Convertirlo en app para App Store / Play Store

Streamlit es una web, no una app nativa. Para llegar a las tiendas hay tres caminos, de menor a mayor esfuerzo:

Camino	Qué implica	Cuándo elegirlo
PWA (Progressive Web App)	Hacer que la web sea "instalable" desde el navegador. La más rápida, pero con presencia limitada en tiendas.	Para validar rápido sin costo.
Wrapper (empaquetar la web)	Envolver la web en un contenedor nativo. Sube a tiendas, pero Apple puede rechazar apps que son "solo una web".	Puente intermedio.
App nativa/híbrida (Flutter, React Native)	Reescribir la interfaz como app real, consumiendo tu mismo backend (Supabase + worker). Lo correcto para producción.	Cuando el MVP ya validó demanda.

Clave para escalar sin rehacer todo: tu **backend ya está listo para escalar**. La base de datos (Supabase) y el worker no cambian; solo cambia la "cara" (de web Streamlit a app nativa). Por eso la separación en planos que hicimos desde el inicio paga aquí: una app nueva consume el mismo Supabase.

Requisitos de tienda: cuenta de Apple Developer (de pago anual) y Google Play Developer (pago único). Ambas exigen política de privacidad y cumplir sus guías de revisión.

14.4 Escalar el motor de datos

- **De Duffel test a Duffel producción** (datos y rutas completas, con costo por uso).
- **Del cron cada 6 h a más frecuencia**, o a un servicio de tareas más robusto (Render, Railway, o funciones programadas) si GitHub Actions se queda corto.
- **Cachear y respetar límites de API** para no exceder cuotas cuando crezcan los usuarios.

15. Cómo monetizar cuando el MVP ya tiene usuarios

Una vez que el MVP está en uso y recogiste feedback de los primeros usuarios, estas son las vías más comunes para un producto de alertas. No son consejos financieros; son modelos habituales que tendrías que validar con tus propios números.

Modelos de ingreso

Modelo	Cómo funciona	Encaje con tu app
Afiliados / referidos	Ganas comisión cuando el usuario compra el vuelo a través de tu enlace	Muy natural: ya rediriges a un checkout. Programas de afiliados de agencias/aerolíneas.
Freemium	Gratis con límites (ej. 2 alertas), plan pago para más alertas / mayor frecuencia / prioridad	Encaja bien: el "valor" es más vigilancia y más rápida.
Suscripción	Cuota mensual por acceso premium (alertas ilimitadas, WhatsApp propio, predicción de precios)	Para usuarios frecuentes (viajeros, cazadores de ofertas).
B2B / API	Vender el servicio de alertas a agencias o empresas	Más adelante, si el motor demuestra ser confiable.

Secuencia recomendada de monetización

1. **Primero, validar valor:** confirma con métricas que la gente crea alertas y abre las notificaciones. Sin uso real, monetizar es prematuro.
2. **Empezar por afiliados:** es el modelo de menor fricción — el usuario no paga, tú ganas por conversión. Ideal para un primer ingreso sin ahuyentar usuarios.
3. **Introducir freemium** cuando tengas features premium claras (predicción de precios, recomendación "comprar ahora vs esperar", WhatsApp con tu marca, alertas ilimitadas).
4. **Suscripción / B2B** cuando el producto esté maduro y tenga usuarios recurrentes.

Ideas de features premium (ya estaban previstas como "mejoras futuras" del proyecto): predicción de precios con machine learning, recomendación de compra, detección de oportunidades por ruta, y priorización de alertas. Cada una es una razón para pagar.

Nota honesta: antes de cobrar, revisa los **términos de uso** de cada API/servicio que reutilizas (Duffel, programas de afiliados, etc.) y los aspectos legales de manejar datos de usuarios (privacidad, consentimiento). Monetizar cambia tus obligaciones. No es consejo legal; conviene revisarlo con alguien especializado cuando llegue el momento.

16. Glosario rápido

Término	Qué es
MVP	Producto mínimo viable: la versión más simple que ya entrega valor y se puede mostrar.
Streamlit	Framework de Python para hacer webs de datos sin escribir HTML/JS.
Supabase	Base de datos PostgreSQL administrada en la nube, con tier gratuito.
Worker / cron	Proceso que corre solo, en horario programado (aquí, cada 6 horas).
GitHub Actions	Servicio de GitHub que ejecuta tareas programadas gratis.
API	Interfaz para pedir datos a un servicio externo (Duffel para vuelos, etc.).
Mock	Proveedor simulado que inventa datos para probar sin costo ni credenciales.
Dry-run	Modo de prueba donde "se hace de cuenta" que se envía, pero solo se registra.
IATA	Código de 3 letras de aeropuertos y ciudades (SCL, MAD, PAR).
.env / Secrets	Donde viven las claves privadas, fuera del código público.
RLS	Row Level Security: control de acceso a filas en Supabase.
Diff	Vista de qué líneas se agregan (verde) y se quitan (rojo) en un cambio de código.

Documento de aprendizaje generado como bitácora del proyecto Flight Price Alert Web ("Somos Rata").
Úsalo como base para replicar, para construir apps nuevas y como referencia de tu portafolio.